

AEGIS AUTOPILOT - LIMITLESS VOID REDESIGN

Master-spec for Codex

Product direction: Arseniy / implementation assessment: Codex

2026-07-22

Table of Contents

1 AEGIS AUTOPILOT — LIMITLESS VOID REDESIGN

Статус документа: master brief / дизайн-концепт / техническое задание / запрос на feasibility review.

Репозиторий: aisarus/aegis-autopilot

Целевая платформа: desktop Electron application, Windows-first.

Ключевое ограничение продукта: Aegis контролирует **не более трёх параллельных ChatGPT-чатов**. Это ограничение не маскируется и не воспринимается как недостаток — оно становится главной художественной метафорой продукта: три чата = три техники = три сферы.

Художественная лицензия: приложение создаётся для частного использования владельцем и не планируется к публичному распространению. В визуальном слое допускается прямое использование образа Сатору Годжо как художественного героя интерфейса. При любом будущем публичном релизе этот вопрос потребуется пересмотреть отдельно.

Главная цель: превратить утилитарный Electron-клиент в живой интерактивный control cockpit, который в состоянии покоя выглядит как премиальная анимированная заставка, а по клику превращается в полноценный интерфейс управления тремя чатами.



Главный экран — *Limitless Void*

2 1. Что именно требуется от Codex

Codex не должен сразу бросаться переписывать приложение. Сначала он должен:

1. Изучить текущий репозиторий, реальный runtime, IPC-контракты, DOM-интеграцию, WebContentsView, тесты и ограничения Electron.
2. Сверить этот документ с фактическим кодом.
3. Разделить требования на:
 - реализуемые непосредственно в текущей архитектуре;
 - реализуемые после локального рефакторинга renderer;
 - требующие новых художественных ассетов;
 - требующие технического proof-of-concept;
 - нежелательные или опасные для производительности/стабильности.
4. Дать честный ответ, **что Codex способен реализовать самостоятельно в репозитории**, а где понадобится внешняя работа: 2D-анимация, Live2D rig, 3D-модель, саунд-дизайн, профессиональные шейдеры или ручная художественная подготовка.
5. Предложить первый маленький PR, который доказывает архитектуру редизайна, но не ломает текущую функциональность.

В конце документа приведён обязательный формат ответа Codex.

3 2. Текущее состояние продукта, которое нельзя потерять

По текущему репозиторию Aegis — Electron-клиент с настоящим ChatGPT, встроенным через отдельный WebContentsView, и локальной панелью управления. В приложении уже существуют:

- максимум три закреплённых проектных чата;
- запуск, остановка и ручная проверка автопилота;
- индивидуальное включение автопилота для каждого чата;
- очередь команд и повторные попытки;
- supervisor на Gemini;
- лимиты Gemini по деньгам и количеству вызовов;
- логирование, диагностика и recovery-механизмы;
- безопасное хранение API-ключа;
- IPC-мост через preload.js;
- renderer на обычных HTML/CSS/JS;
- smoke-тесты на основные гарантии работы;
- встроенная настоящая страница ChatGPT, а не визуальная имитация.

3.1 2.1. Неприкосновенные продуктовые гарантии

Редизайн **не имеет права**:

- сломать вход в ChatGPT;
- менять контракт отправки сообщений без отдельной задачи;
- создавать дублирующие отправки;
- ослаблять защиту черновика;
- ломать привязку команды к конкретному чату;
- убирать бюджетные ограничения;
- снижать качество диагностики;
- смешивать состояния трёх чатов;
- делать красивую анимацию важнее подтверждённого выполнения команды.

3.2 2.2. Основной принцип внедрения

Новая оболочка строится **вокруг существующей функциональности**, а не вместо неё.

Сначала создаётся новый renderer и визуальный shell, который читает тот же публичный state и вызывает те же IPC-методы. Только после доказательства parity можно менять внутреннюю архитектуру.

4 3. Главная продуктовая метафора

Aegis — это не dashboard с тремя карточками. Aegis — это Сатору Годжо, удерживающий три независимых интеллектуальных процесса внутри Limitless Void.

- **Blue** — чат 1. Исследование, анализ, сбор фактов, спокойная концентрация.
- **Red** — чат 2. Выполнение, стратегия, операции, риск, действие.
- **Hollow Purple** — чат 3. Синтез, творчество, сложная интеллектуальная работа, трансформация.
- **Повязка Годжо** — доступ к мета-уровню: настройки, контракты, диагностика и правила поведения системы.

Пользователь не видит постоянное меню. В обычном состоянии приложение должно выглядеть как живая заставка на большом мониторе. Все рабочие экраны скрыты за интерактивными объектами сцены.

5 4. Информационная архитектура

5.1 4.1. Домашний экран

Видимы только:

- Годжо;
- Limitless Void;
- три техники/сферы;
- минимальные hover-подсказки;
- едва заметный системный статус;
- скрытые hotspot-зоны.

Никаких постоянных боковых панелей, метрик, больших кнопок или таблиц.

5.2 4.2. Три пространства чатов

Каждая сфера открывает отдельное рабочее пространство, но все три используют один и тот же базовый layout и разные цветовые/семантические модификаторы.

- Click Blue → Chat Blue workspace.
- Click Red → Chat Red workspace.
- Click Purple → Chat Purple workspace.

5.3 4.3. Повязка как системный портал

Клик по повязке открывает системный hub. Из него доступны:

- настройки конкретного чата;
- глобальные настройки;
- контракты и политики;
- диагностика и логи;
- управление графикой и анимациями;
- возврат на домашнюю сцену.

5.4 4.4. Скрытая навигация

На рабочих экранах может появляться компактная навигация, но она должна исчезать при возврате домой. Нельзя переносить dashboard-хром на домашнюю заставку.

6 5. Атлас референсов

6.1 5.1. Главный экран — Limitless Void



Home / Limitless Void

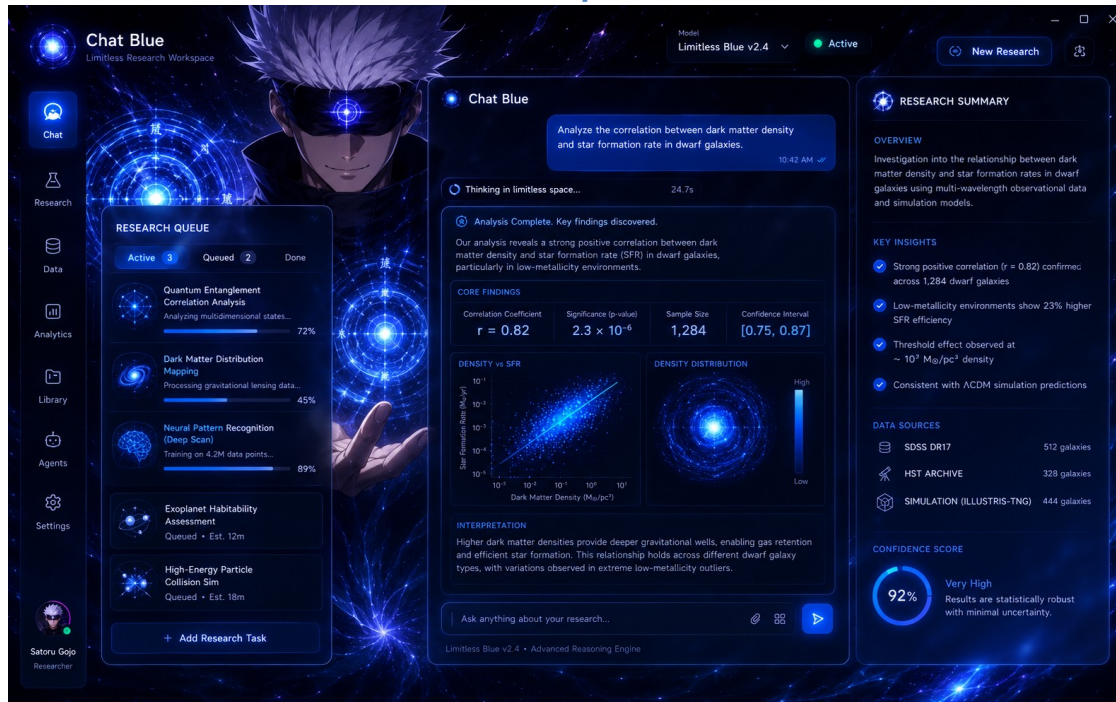
6.1.1 Что обязательно сохранить

- Годжо занимает центр и является главным смысловым объектом.
- Фон обладает большой глубиной, а не выглядит как плоская обои.
- Blue, Red и Purple заметно различаются по физике света.
- Повязка имеет тонкий hotspot, но не превращается в кнопку.
- Никакого обычного интерфейса поверх заставки.

6.1.2 Что улучшить при реализации

- Сферы должны быть ближе к каноническим техникам, а не к планетам.
- Blue должен визуально втягивать материю внутрь.
- Red должен расширять/выталкивать энергию наружу.
- Purple должен ощущаться как исчезновение пространства, а не просто фиолетовый шар.
- Сцена должна работать в idle-режиме часами, не перегружая GPU.

6.2 5.2. Chat Blue — research workspace



Chat Blue

6.2.1 Смысл

Blue — холодная аналитическая среда. Она не должна быть «синим dashboard». Она должна ощущаться как вход внутрь техники притяжения, где данные, источники и задачи собираются к центру.

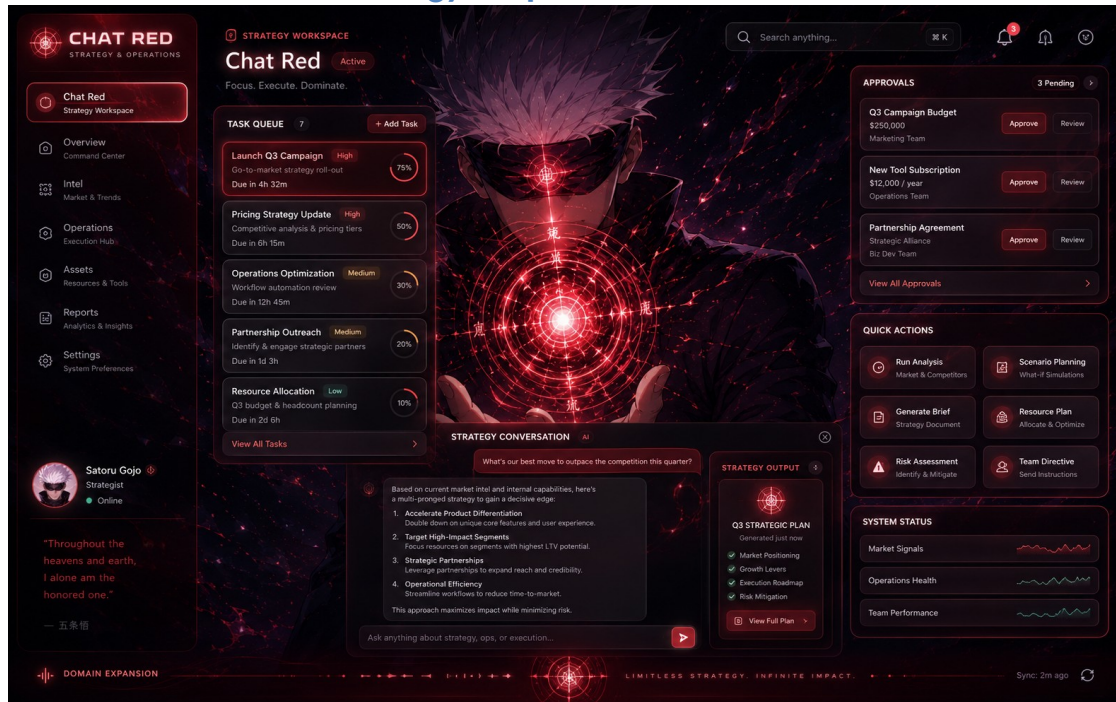
6.2.2 Сохраняем

- research queue;
- основную область настоящего ChatGPT;
- краткий статус чата;
- источники/метрики;
- синюю энергетическую геометрию.

6.2.3 Убираем или упрощаем

- декоративные научные данные, которые не существуют в приложении;
- лишние постоянные панели;
- фальшивые модельные селекторы в основном рабочем режиме.

6.3 5.3. Chat Red — strategy & operations



Chat Red

Red — пространство действия. Оно связано с очередью, решением supervisor, подтверждениями, следующей командой и рисками.

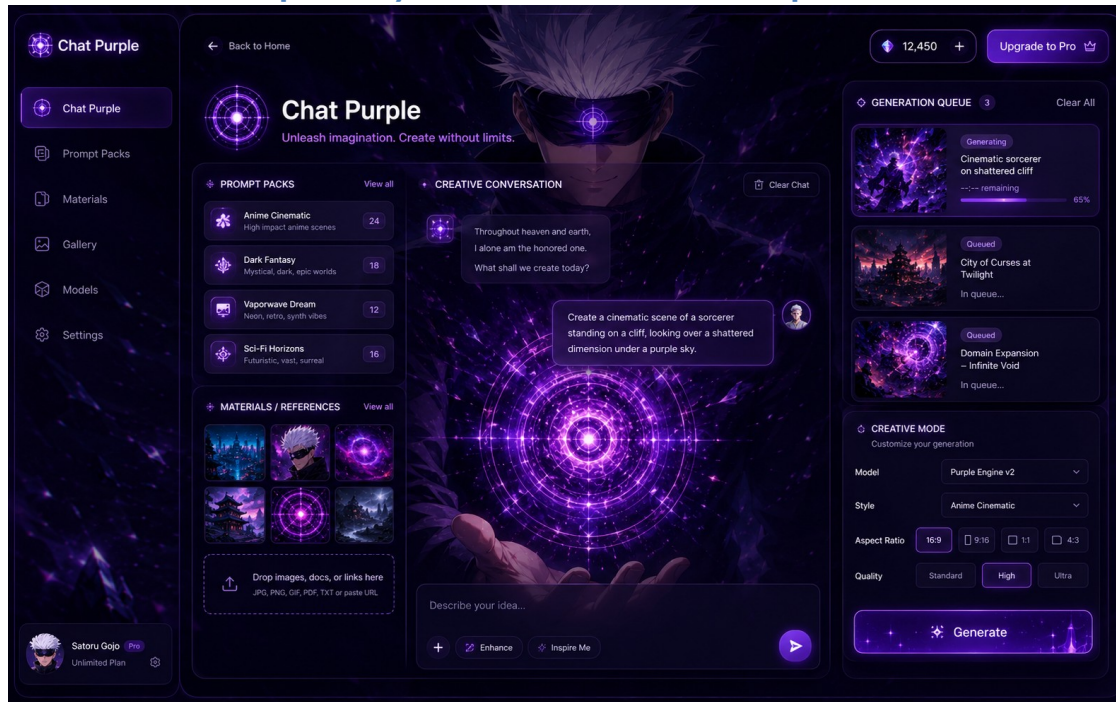
6.3.1 Сохраняем

- сильную красную идентичность;
- приоритеты и очередь;
- approvals;
- next action;
- ощущение давления и энергии наружу.

6.3.2 Важное ограничение

Интерфейс не должен создавать впечатление, что Aegis самостоятельно выполняет операции, которых в реальном runtime нет. Все визуальные статусы должны отображать существующее состояние.

6.4 5.4. Chat Purple — synthesis & creative workspace



Chat Purple

Purple — наиболее магическое пространство, но оно не должно превращать приложение в генератор картинок. В реальном Aegis это третий ChatGPT-чат; его визуальная тема может означать творчество, глубокий синтез или наиболее сложный проект.

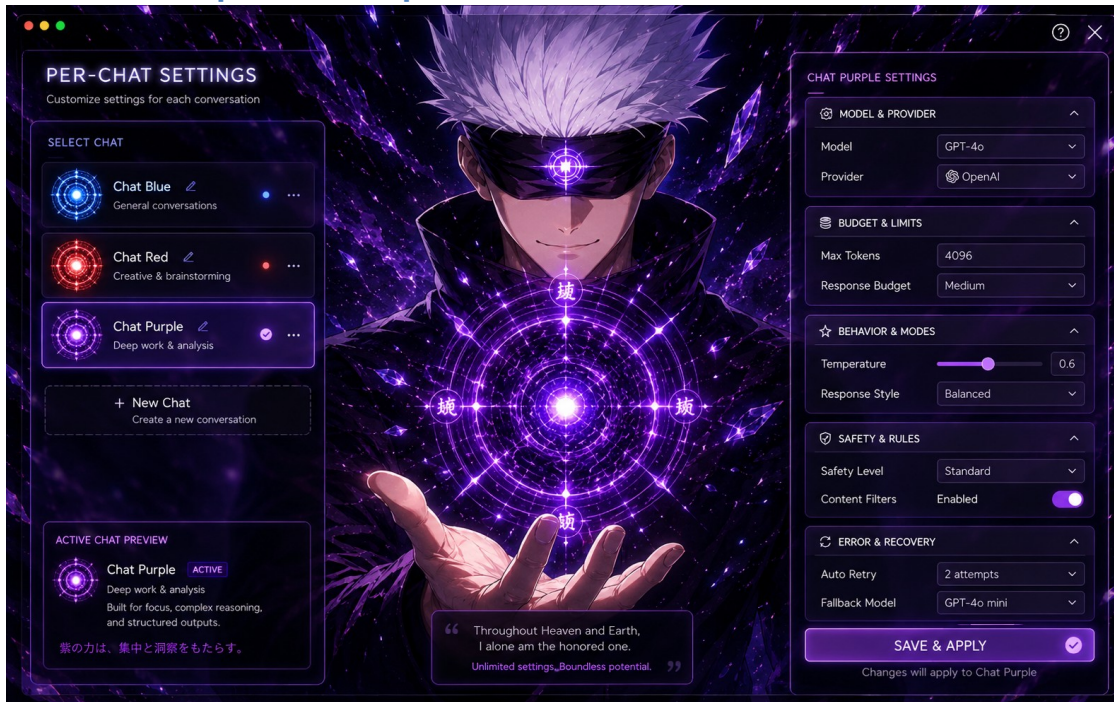
6.4.1 Сохраняем

- портал Hollow Purple;
- материалы и документы;
- очередь;
- контракты/инструкции;
- насыщенную фиолетовую среду.

6.4.2 Адаптируем

Названия блоков должны формироваться из реальных функций Aegis: чат, очередь, документы, история действий, supervisor, лимиты и статус.

6.5 5.5. Настройки конкретного чата

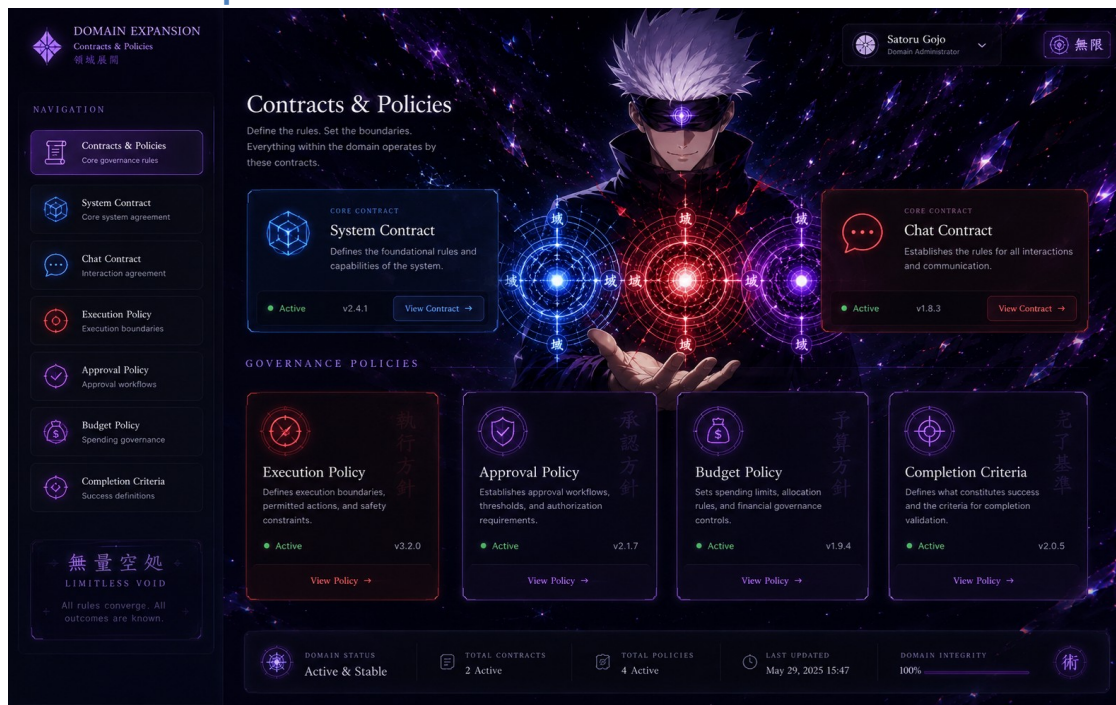


Per-chat settings

Экран должен восприниматься как вскрытие и настройка энергетического ядра выбранной сферы.

Слева — выбор одного из трёх чатов. В центре — живая сфера. Справа — нормальные редактируемые настройки.

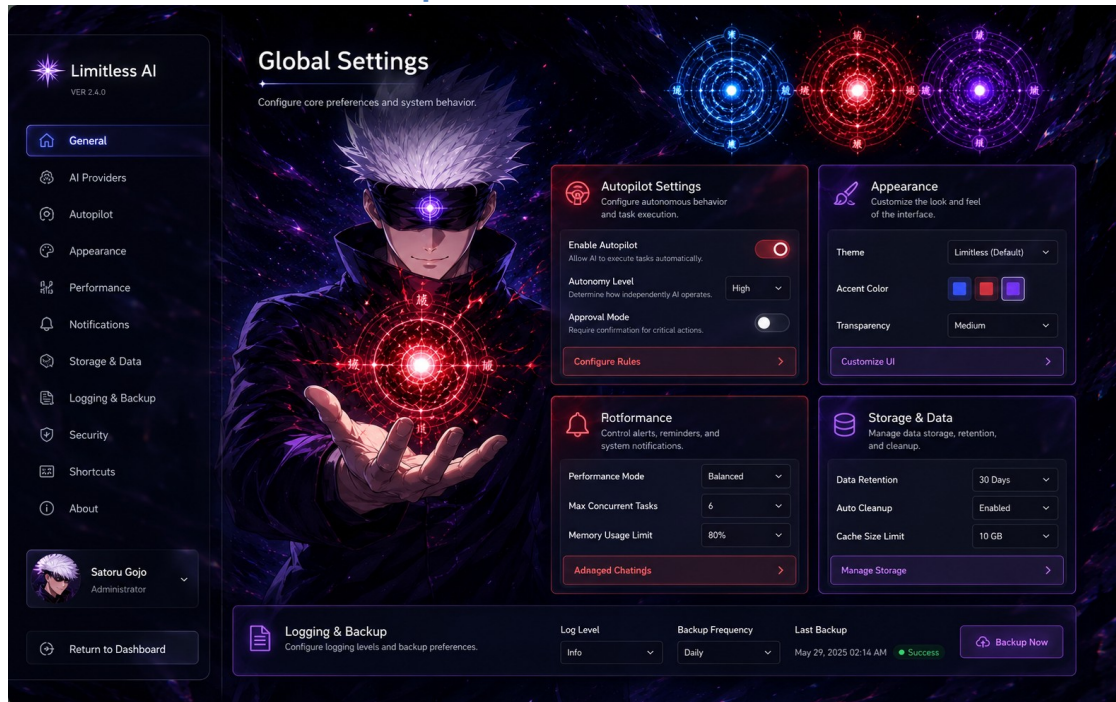
6.6 5.6. Контракты и политики



Contracts and policies

Этот экран открывается как Domain Expansion. Он показывает иерархию правил, но остаётся редактируемым рабочим интерфейсом.

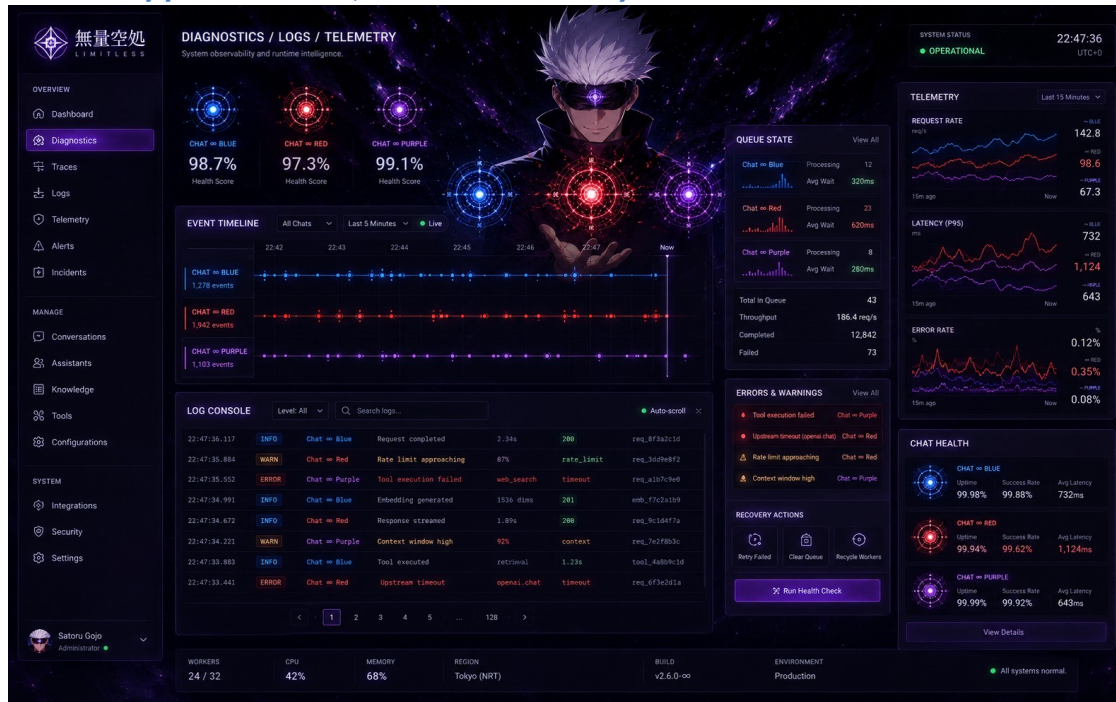
6.7 5.7. Глобальные настройки



Global settings

Глобальные настройки менее театральны. Здесь важнее читаемость, безопасность ключей и контроль качества графики.

6.8 5.8. Диагностика, логи и telemetry



Diagnostics

Диагностика должна быть самой технической частью продукта. Здесь допускается высокая плотность информации, но она обязана отражать реальные события и real runtime.

7 6. Художественная система

7.1 6.1. Общий тон

Не «аниме-скин поверх админки», а единая интерактивная сцена.

Ключевые слова:

- limitless;
- void;
- cosmic obsidian;
- elegant destructive energy;
- anime key art;
- premium sci-fi instrument;
- controlled spectacle;
- hidden complexity.

7.2 6.2. Годжо

Годжо должен быть не картинкой-фоном, а частью состояния приложения.

7.2.1 Idle-анимации

- почти незаметное дыхание;
- движение воротника и ткани;
- тонкое колебание волос;
- микродвижение головы за курсором в пределах 1–2 градусов;
- свет от сфер скользит по лицу и одежде;
- частицы замедляются, когда мышь неподвижна;
- повязка очень редко даёт слабый импульс Six Eyes.

7.2.2 Состояния

- all idle: спокойная улыбка, плавная камера;
- one chat active: соответствующий цвет сильнее освещает лицо;
- multiple chats active: три света балансируются;
- error: улыбка исчезает, частицы становятся резче, но без дешёвого красного мигания;
- budget limit: вокруг соответствующей сферы появляется ограничивающее кольцо;
- all tasks done: короткая спокойная кульминация, затем сцена возвращается в idle.

7.3 6.3. Limitless Void

Фон должен быть многослойным:

1. дальний бесконечный космос;
2. медленные туманности;
3. пространство с эффектом искажения;
4. средние кристаллические осколки;
5. близкие частицы;
6. редкие световые импульсы;
7. мягкий vignette.

Нельзя использовать постоянный сильный шум и одинаково быстрые частицы — это быстро утомляет.

7.4 6.4. Blue

Blue не просто синий цвет.

Визуальная физика:

- частицы втягиваются в ядро;

- кольца сжимаются;
- фон около сферы слегка деформируется;
- при hover близлежащие осколки смещаются к центру;
- звуковой образ — низкий втягивающий резонанс;
- переход в экран — камера ускоряется внутрь точки притяжения.

7.5 6.5. Red

Red — инверсия и выброс.

- кольца расширяются;
- частицы разлетаются наружу;
- свет даёт короткую ударную волну;
- при hover поверхность сферы кажется переполненной энергией;
- переход — сфера сначала отталкивает камеру назад, затем резко раскрывает пространство.

7.6 6.6. Hollow Purple

Purple — не смесь цветов в лоб, а удаление/разрыв пространства.

- вокруг ядра исчезают частицы;
- виден мягкий negative-space tunnel;
- Blue и Red кратко просматриваются как два потока внутри Purple;
- края сферы преломляют фон;
- переход — камера проходит через область, где изображение почти полностью теряет форму, затем собирается в новый экран.

7.7 6.7. Материалы интерфейса

- базовый фон панелей: почти чёрный с фиолетово-синим оттенком;
- прозрачность только при достаточной подложке;
- blur используется ограниченно;
- границы — тонкие, без неоновой обводки вокруг каждого элемента;
- glow только для активных состояний;
- карточки не должны спорить с персонажем;
- в рабочих экранах арт затемняется до 20–35% визуального веса.

7.8 6.8. Типографика

Рекомендуемый принцип:

- UI: Inter / Geist / Manrope / system fallback;
- крупные художественные заголовки: более характерный display font, но без псевдояпонской карикатуры;
- monospace: JetBrains Mono для логов;

- японские подписи используются как декоративный слой, а не как основной источник смысла.

7.9 6.9. Иконография

Иконки должны объединять:

- простую геометрию интерфейса;
 - круговые техники;
 - три цветовых сигнатуры;
 - малую толщину линий;
 - понятность без текста.
-

8 7. Главный экран: подробнейшая спецификация

8.1 7.1. Композиция

- Годжо расположен строго по центру или с лёгким смещением вверх.
- Лицо находится в верхней трети экрана.
- Blue слева, Red по центру, Purple справа.
- Сферы не закрывают лицо и не выглядят одинаковыми.
- Центральная ладонь может поддерживать Red; две другие сферы удерживаются композицией/энергией.
- Низ экрана остаётся достаточно тёмным для системных micro-indicators.

8.2 7.2. Интерактивные зоны

8.2.1 Сферы

- hotspot больше визуальной сферы на 12–20%;
- курсор слегка притягивается к сфере;
- появляется тонкое название чата и статус;
- сфера поворачивается к камере;
- соответствующий chat state кратко проявляется вокруг ядра;
- single click открывает чат;
- right click или long press открывает быстрые действия;
- middle click запрещён или используется только при явном решении.

8.2.2 Повязка

- hover вызывает мягкое раскрытие кругового символа;
- клик активирует Six Eyes transition;
- никакой обычной кнопки «Settings» на главном экране;
- должна существовать клавиатурная альтернатива, например Ctrl+.,.

8.2.3 Скрытые системные показатели

В нижнем углу допускается едва видимая строка:

- онлайн/оффлайн;
- активных чатов 2/3;
- Gemini budget status;
- текущий time.

Она становится читаемой только при hover или при проблеме.

8.3 7.3. Idle как живые обои

Приложение должно оставаться красивым без взаимодействия:

- 30–60 FPS в foreground в зависимости от режима;
 - 1–5 FPS или пауза в background;
 - медленный camera drift;
 - никаких повторяющихся заметных циклов короче 20 секунд;
 - частицы распределяются процедурно;
 - light flicker очень редкий;
 - случайные микрособытия не должны отвлекать от работы за компьютером.
-

9 8. Киношные переходы

9.1 8.1. Boot sequence

Длительность первого запуска: 3–5 секунд. Повторные запуски — короткий вариант 1–1.5 секунды.

Сценарий:

1. Чёрный экран.
2. Слабый звук пустоты.
3. В центре возникает точка Six Eyes.
4. Из точки расходится тонкая сетка пространства.
5. В темноте проявляется силуэт Годжо.
6. Blue и Red рождаются отдельно.
7. Purple формируется последним как короткая кульминация.
8. Камера мягко отъезжает к рабочей композиции.
9. Hotspot-зоны активируются только после стабилизации сцены.

9.2 8.2. Hover сферы

Длительность 250–450 мс.

- сфера выдвигается на 3–5% к камере;
- depth of field смещается на неё;
- остальные техники слегка затихают;
- появляется краткий label: название чата + состояние;
- Годжо переводит минимальное внимание в её сторону;
- при уходе курсора всё плавно возвращается.

9.3 8.3. Вход в Blue

1. Blue начинает втягивать частицы.
2. Камера движется по мягкой дуге влево.
3. Сцена вокруг растягивается к центру Blue.
4. Домашний UI снимается как texture/snapshot.
5. Сфера раскрывается как тоннель.
6. В тоннеле появляются линии настоящего чата.
7. Камера выходит в Blue workspace.
8. Настоящий WebContentView появляется после завершения маскировочного кадра.

Целевая длительность: 900–1400 мс. Быстрый режим: 450–650 мс.

9.4 8.4. Вход в Red

1. Red сжимается на 100–150 мс.
2. Идёт короткий энергетический выброс.
3. Камеру отталкивает назад.
4. Пространство переворачивается через красный shockwave.
5. Карточки Red workspace собираются из разлетающихся фрагментов.

9.5 8.5. Вход в Purple

1. На мгновение Blue и Red дают два встречных потока.
2. В центре Purple образуется область отсутствия изображения.
3. Частицы пропадают при приближении к ядру.
4. Камера входит в negative-space tunnel.
5. Интерфейс появляется через мягкий purple dissolve.

9.6 8.6. Переключение между чатами

Полный возврат домой при каждом переключении красив, но утомителен. Поэтому нужны два режима:

- **Cinematic switching:** короткий отлёт к центральной сцене и вход в другую сферу;

- **Rapid switching:** цветовой spatial wipe 350–500 мс с кратким появлением орбит трёх сфер.

По умолчанию после первых трёх полных переходов используется rapid switching.

9.7 8.7. Открытие повязки

Самый выразительный переход приложения.

1. Остальные объекты теряют насыщенность.
2. Камера приближается к лицу.
3. Ткань повязки получает тонкое движение, но не обязательно физически снимается.
4. В центре появляется Six Eyes aperture.
5. Из апертуры раскрывается системная геометрия.
6. Пользователь попадает в Settings Hub.

В ultra-версии повязка может частично раздвигаться, открывая глаз. Для этого нужен отдельный качественный ассет/rig. Без ассета используется abstract aperture transition.

9.8 8.8. Contracts: Domain Expansion

1. Текущий экран замирает на 50–80 мс.
2. Панели уходят в глубину.
3. Пространство становится бесконечным.
4. Карточки контрактов появляются как плавающие slabs.
5. Текст остаётся обычным DOM, а глубина создаётся сценой позади.
6. Выбранный контракт приближается и становится редактором.

9.9 8.9. Diagnostics transition

- художественный фон теряет яркость;
- три сферы уменьшаются до health indicators;
- линии частиц выпрямляются в timeline;
- cosmic noise превращается в telemetry traces;
- переход должен ощущаться как переход от магии к инженерному слою.

9.10 8.10. Возврат домой

- рабочие панели растворяются или складываются;
 - snapshot настоящего ChatGPT втягивается обратно в сферу;
 - камера возвращается в hero position;
 - если чат активен, его сфера продолжает пульсировать.
-

10 9. Спецификация рабочих экранов

10.1 9.1. Общий принцип

Настоящий ChatGPT остаётся основным содержимым. Aegis не должен полностью заменять его фальшивой чат-лентой.

Рекомендуемый layout:

- узкая Aegis-панель или overlay на одной стороне;
- WebContentView занимает основной объём;
- дополнительные панели открываются по demand;
- фон и сфера видны в свободных зонах, но не мешают тексту.

10.2 9.2. Chat Blue

10.2.1 Основные блоки

- статус чата;
- автопилот on/off;
- очередь;
- последняя стабильная реакция supervisor;
- бюджет/вызовы;
- документы;
- открыть настоящий чат;
- настройки чата.

10.2.2 Motion language

- элементы появляются к центру;
- progress bars сжимаются/собираются;
- активное состояние — мягкая концентрация частиц;
- ошибки — нарушение орбит, а не красная сирена.

10.3 9.3. Chat Red

10.3.1 Основные блоки

- активная команда;
- очередь;
- решение supervisor;
- approval, если действие требует пользователя;
- причина паузы;
- retry/recovery;
- next action.

10.3.2 Motion language

- action states раскрываются наружу;

- отправка команды — короткий controlled pulse;
- loop detected — сфера создаёт три одинаковых импульса и блокируется кольцом;
- paused — энергия удерживается защитным контуром.

10.4 9.4. Chat Purple

10.4.1 Основные блоки

- чат;
- документы/материалы;
- extracted prompt pack;
- очередь;
- review/continue decisions;
- сложные статусы проекта.

10.4.2 Motion language

- dissolve;
 - пространственное смещение;
 - более медленные и сложные transitions;
 - отсутствие агрессивного постоянного glow.
-

11 10. Настройки конкретного чата

11.1 10.1. Поля

Не выдумывать настройки, которых runtime не поддерживает. Базовый список должен строиться от фактического state:

- отображаемое имя;
- привязанный URL/идентификатор чата;
- автопилот включён;
- текущий autopilot state;
- pause reason;
- supervisor mode;
- cooldown;
- max auto turns;
- лимит вызовов;
- лимит расходов;
- поведение queue;
- контракт/инструкция;
- режим review/auto;
- reset counters, если безопасно;

- remove chat.

11.2 10.2. Визуальная идея ядра

В центре находится core выбранного чата. При изменении настройки:

- кольца перестраиваются;
- интенсивность соответствует степени автономности;
- budget limit меняет внешний containment ring;
- review mode добавляет защитный слой;
- unsafe/invalid configuration визуально конфликтует, но сохранение блокируется обычной validation logic.

11.3 10.3. UX

- все контролы доступны с клавиатуры;
 - значения подписаны явно;
 - анимация не заменяет текстовое подтверждение;
 - Save & Apply показывает diff изменённых полей;
 - destructive actions требуют явного подтверждения.
-

12 11. Контракты и политики

12.1 11.1. Типы контрактов

- Global/System Contract;
- Chat Contract;
- Execution Policy;
- Approval Policy;
- Budget Policy;
- Completion Criteria;
- Recovery Policy;
- Loop Protection;
- Privacy/Data Sharing Policy.

12.2 11.2. Наследование

Codex должен предложить понятную модель:

Global defaults → Chat overrides → Runtime safety hard limits

Hard limits не могут быть отключены художественным интерфейсом.

12.3 11.3. Редактор

- обычный текстовый редактор или CodeMirror/Monaco;

- syntax-like подсветка секций;
- version history;
- diff до/после;
- preview эффективного контракта;
- конфликтующие правила;
- безопасное сохранение;
- экспорт/импорт.

12.4 11.4. Domain визуализация

Плавающие карточки — только навигационный слой. Редактирование должно происходить в стабильной плоскости, чтобы текст не двигался в 3D.

13 12. Глобальные настройки

Категории:

- Gemini API / secure storage status;
- model;
- глобальные budget limits;
- cooldown defaults;
- графический профиль;
- cinematic transitions;
- reduced motion;
- audio;
- notifications;
- logging;
- data retention;
- cache;
- backup/export;
- privacy;
- diagnostics collection.

13.1 12.1. Графические профили

13.1.1 Ultra / Cinematic

- полноценные particles;
- bloom;
- depth of field;
- distortion shaders;
- high-resolution character assets;
- dynamic lighting;

- 60 FPS target.

13.1.2 Balanced

- упрощённые частицы;
- ограниченная постобработка;
- 30/60 adaptive FPS;
- статичный character base с небольшим parallax.

13.1.3 Performance

- pre-rendered background video или layered 2D;
- CSS/SVG spheres;
- минимальные переходы;
- отключение непрерывного WebGL render loop.

13.1.4 Reduced Motion

- fades и cross-dissolves;
 - без пролётов камеры;
 - без screen shake;
 - без depth-of-field pumping;
 - все функции доступны.
-

14 13. Диагностика и визуальная телеметрия

14.1 13.1. Источники данных

Показывать только то, что реально существует:

- browser status;
- signed in;
- current URL/chat;
- composer ready;
- generating;
- conversation ready;
- queue length;
- scheduler busy;
- pending decisions;
- usage today;
- API key configured;
- last error;
- events;
- retries;
- last processed response.

Не подставлять фальшивый CPU, RAM, network throughput, если приложение их не собирает.

14.2 13.2. Визуальная трансформация

- события Blue/Red/Purple получают три цветовые дорожки;
 - recovery — зелёный нейтральный цвет;
 - errors — не только красный, но и конкретная иконка/текст;
 - hover на событии подсвечивает соответствующую сферу;
 - click открывает подробности и безопасную копию diagnostics.
-

15 14. Маппинг runtime state → визуальное состояние

Runtime state	Главный экран	Рабочий экран	Звук
off	сфера почти спокойна	выключенный core	тишина
watching	медленная орбита	мягкий pulse	слабый hum
checking	частицы сходятся	progress ring	короткий scan
waiting-response	ядро стабильно светится	waiting state	редкий pulse
reviewing	дополнительное кольцо	review overlay	двойной тон
decision-ready	hotspot становится ярче	decision card	мягкий chime
needs-user	сфера слегка приближается	prominent question	уведомление
paused	containment ring	pause reason	приглушение
error	нарушенная орбита	error details	короткий low impact
loop-detected	три повторных импульса, затем lock	loop warning	тройной сигнал
done	спокойная стабилизация	completion state	короткий resolve

Это таблица намерения. Codex должен сопоставить её с фактическими строковыми состояниями в репозитории и предложить точный enum.

16 15. Предлагаемая техническая архитектура

16.1 15.1. Renderer

Рекомендуемый стек:

- React;
- TypeScript;
- Vite;
- Zustand или XState;
- React Three Fiber / Three.js;
- GSAP для режиссируемых timeline;
- Framer Motion для DOM micro-interactions;
- CSS variables/design tokens;
- optional CodeMirror для контрактов.

Codex может предложить более простой стек, если докажет, что он лучше вписывается в существующий репозиторий.

16.2 15.2. Слои UI

1. SceneLayer — WebGL/Canvas, Годжо, Void, сферы, частицы, камера.
2. TransitionLayer — snapshots, masks, temporary portal effects.
3. AppShell — DOM навигация и рабочие панели.
4. WebContentView — настоящий ChatGPT.
5. ModalLayer — settings, contracts, confirmations.
6. Toast/AlertLayer — критические сообщения.

16.3 15.3. State boundary

Main process остаётся источником правды. Renderer получает сериализуемый public state через IPC.

Предложить:

```
interface AegisVisualState {  
  activeChatId: string | null;  
  screen: 'home' | 'chat' | 'chat-settings' | 'contracts' | 'global-settings' | 'diagnostics';  
  transition: TransitionState;  
  graphicsProfile: 'ultra' | 'balanced' | 'performance';  
  reducedMotion: boolean;  
}
```

Это UI-state, а не замена runtime state.

16.4 15.4. Animation state machine

Нужна явная state machine:

HOME_IDLE
HOME_HOVER_BLUE | HOME_HOVER_RED | HOME_HOVER_PURPLE |
HOME_HOVER_BLINDFOLD
ENTERING_CHAT
CHAT_READY
SWITCHING_CHAT
OPENING_SYSTEM_HUB
SYSTEM_SCREEN_READY
RETURNING_HOME
RECOVERING_TRANSITION

Переходы блокируют конфликтующие клики и всегда имеют fallback timeout.

16.5 15.5. WebContentsView cinematic bridge

Это критический proof-of-concept.

Поскольку WebContentsView не является обычным DOM-элементом, его нельзя свободно вращать CSS-трансформациями внутри renderer. Предлагаемый мост:

1. Получить capturePage() содержимого view.
2. Передать изображение в renderer как временный data URL / native image bridge.
3. Отобразить snapshot в TransitionLayer.
4. Скрыть реальный view.
5. Проиграть анимацию snapshot.
6. Изменить bounds реального view.
7. Дождаться стабильного layout.
8. Показать view и убрать snapshot.

Codex должен подтвердить, насколько это надёжно в Electron 39 и какие ограничения по задержке/памяти возникают.

16.6 15.6. Bounds synchronization

При resize, DPI change и переходах:

- renderer вычисляет целевую область;
- IPC передаёт bounds в main;
- main обновляет WebContentsView;
- animation завершает только после layout ack;
- при ошибке view появляется без анимации.

16.7 15.7. Security

- contextIsolation: true сохраняется;
 - Node не доступен renderer;
 - API-ключ не передаётся в UI;
 - список IPC методов allowlisted;
 - динамический HTML из ChatGPT не копируется в renderer;
 - snapshots не сохраняются на диск без явного действия;
 - внешние ссылки открываются через безопасный existing handler;
 - CSP усиливается после перехода на Vite.
-

17 16. Художественные ассеты и варианты реализации Годжо

17.1 16.1. Вариант А — layered 2.5D (рекомендуемый)

Нужны слои:

- дальний фон;
- ближний Void;
- тело;
- голова;
- волосы перед/зад;
- повязка;
- лицо;
- рука;
- световые маски Blue/Red/Purple;
- foreground shards.

Анимации:

- parallax;
- skeletal deformation отдельных частей;
- light masks;
- небольшая head tracking;
- breathing loop.

Плюсы: анимешный вид, хорошая производительность, реалистичный объём работы.

17.2 16.2. Вариант В — Live2D/Spine

Потребуется внешний rig. Codex сможет интегрировать runtime, но не должен обещать профессионально разрезать и заригать персонажа без исходных художественных слоёв.

17.3 16.3. Вариант С — полноценный 3D

Самый рискованный вариант. Нужны:

- качественная модель;
- toon shader;
- facial rig;
- hair rig;
- cloth;
- правильная камера;
- лицензированный/подготовленный asset.

Без профессионального 3D-ассета результат будет хуже 2.5D.

17.4 16.4. Вариант D — pre-rendered video wallpaper

Для MVP можно использовать:

- короткий seamless WebM/MP4 loop;
- отдельно интерактивные сферы в WebGL;
- DOM hotspots поверх;
- видео для idle, real-time transitions для кликов.

Это снижает сложность и даёт быстрое визуальное качество.

18 17. Сферы: технический дизайн

18.1 17.1. Общая структура

Каждая сфера состоит из:

- core shader;
- 2–4 орбитальных колец;
- particle emitter;
- distortion layer;
- hover hit volume;
- state halo;
- label anchor;
- status mapping.

18.2 17.2. Blue shader

- inward flow field;
- radial compression;
- refraction к центру;

- particles attracted via curl noise + inverse radius;
- мягкая синяя каустика на персонаже.

18.3 17.3. Red shader

- outward flow;
- emissive pulse;
- shockwave ring;
- краткие sparks;
- controlled chromatic edge.

18.4 17.4. Purple shader

- signed distance field / mask hole;
- background displacement;
- particle culling near center;
- смещение blue/red light;
- более тёмное ядро и яркий edge.

18.5 17.5. Fallback

Если custom shaders слишком рискованны:

- animated SVG rings;
- sprite sheets;
- CSS radial gradients;
- canvas particles;
- pre-rendered transparent WebM spheres.

Codex должен определить минимальный proof, который показывает ощущение техники без преждевременной shader-архитектуры.

19 18. Саунд-дизайн

Звук не обязателен для первой версии, но архитектура должна его допускать.

- ambient void loop — очень тихий;
- Blue hover — втягивающий sub tone;
- Red hover — короткое давление;
- Purple — почти без атаки, с пространственным провалом;
- blindfold — чистый high-frequency aperture;
- success — короткое спокойное разрешение;
- error — неброский низкий импульс;
- все звуки отключаемы;

- общий limiter, чтобы эффекты не были громче системных уведомлений.
-

20 19. Производительность

20.1 19.1. Бюджеты

Целевые бюджеты для Balanced:

- idle GPU frame: до 8–10 мс;
- active transition: до 16.6 мс на 60 FPS;
- particle count: адаптивный;
- texture memory: контролируемая, без необоснованных 4К слоёв;
- snapshot lifetime: минимальный;
- blur passes: не более необходимых;
- renderer pauses when hidden.

20.2 19.2. Adaptive quality

Качество автоматически снижается, если:

- средний FPS ниже порога;
- GPU/renderer frame time нестабилен;
- окно не в фокусе;
- включён battery saver;
- аппаратное ускорение недоступно.

20.3 19.3. Деградация без потери функций

При выключенном WebGL:

- home = статичное изображение + CSS hover;
 - screens = обычный DOM;
 - transitions = fade;
 - все действия доступны.
-

21 20. Accessibility и управление

- полная клавиатурная навигация;
- visible focus ring;
- сферы имеют скрытые доступные кнопки;
- screen reader labels: Open Chat Blue, Open Chat Red, Open Chat Purple, Open system controls;

- reduced motion;
 - high contrast;
 - не полагаться только на цвет;
 - ошибки описываются текстом;
 - Esc возвращает/закрывает overlay;
 - shortcut help доступен из системного hub.
-

22 21. План внедрения

22.1 Phase 0 — Audit and feasibility

Цель: понять текущую архитектуру и подтвердить путь без изменения поведения.

Deliverables:

- карта файлов и runtime boundaries;
- список IPC;
- точные состояния чатов;
- карта WebContentView layout;
- тестовый план;
- capability matrix;
- решение: React/Vite или эволюция текущего renderer.

Acceptance:

- никакого изменения пользовательского поведения;
- все smoke tests проходят.

22.2 Phase 1 — Design system shell

- TypeScript renderer scaffold;
- design tokens;
- route/screen state;
- IPC adapter;
- static home scene с текущим PNG;
- три доступных hotspot-кнопки;
- blindfold hotspot;
- простой fade navigation;
- сохранение текущей функциональности.

22.3 Phase 2 — Home scene MVP

- layered background;
- Three.js canvas или lightweight equivalent;

- three animated spheres;
- hover states;
- real runtime status mapping;
- graphics profile;
- pause when unfocused;
- screenshots and visual regression.

22.4 Phase 3 — WebContentView transition POC

- capturePage;
- hide/show;
- animate snapshot;
- bounds synchronization;
- recovery timeout;
- test on Windows scaling 100%, 125%, 150%.

Этот этап является go/но-go для самых кинематографичных переходов.

22.5 Phase 4 — Chat workspaces

- общий workspace template;
- Blue/Red/Purple themes;
- existing control actions;
- responsive panel sizing;
- real ChatGPT remains central;
- fast switching;
- state-specific visuals.

22.6 Phase 5 — Per-chat settings

- actual fields only;
- validation;
- dirty state;
- save/apply;
- visual core mapping;
- safe destructive actions.

22.7 Phase 6 — Contracts and policies

- data model;
- editor;
- effective contract preview;
- export/import;
- Domain transition;
- tests for persistence.

22.8 Phase 7 — Global settings and graphics profiles

- API status;
- graphics;
- reduced motion;
- audio;
- storage/logging;
- migration of existing settings.

22.9 Phase 8 — Diagnostics

- real events;
- timelines;
- filters;
- copy diagnostics;
- recovery actions;
- visual mapping to spheres.

22.10 Phase 9 — Cinematic polish

- GSAP timelines;
- postprocessing;
- camera choreography;
- audio hooks;
- micro-animation;
- boot sequence;
- screenshot/video capture for QA.

22.11 Phase 10 — Stabilization

- performance profiling;
 - crash recovery;
 - GPU fallback;
 - DPI tests;
 - long idle soak test;
 - memory leak test;
 - keyboard/ally;
 - installer validation;
 - final smoke suite.
-

23 22. Рекомендуемая структура кода

Codex должен адаптировать её к реальности, а не копировать буквально.

- renderer/
 - app/
 - App.tsx
 - routes.ts
 - visual-state.ts
 - bridge/
 - aegis-api.ts
 - types.ts
 - scene/
 - AegisScene.tsx
 - CameraRig.tsx
 - GojoLayer.tsx
 - LimitlessVoid.tsx
 - techniques/
 - BlueTechnique.tsx
 - RedTechnique.tsx
 - PurpleTechnique.tsx
 - transitions/
 - TransitionDirector.ts
 - timelines.ts
 - SnapshotPlane.tsx
- screens/
 - HomeScreen.tsx
 - ChatWorkspace.tsx
 - PerChatSettings.tsx
 - ContractsScreen.tsx
 - GlobalSettings.tsx
 - DiagnosticsScreen.tsx
- components/
 - GlassPanel.tsx
 - StatusBadge.tsx
 - ChatSelector.tsx
 - BudgetMeter.tsx
 - DecisionCard.tsx
- styles/
 - tokens.css
 - themes.css
 - motion.css
- main/
 - view-layout.js
 - capture-transition.js
 - ipc-handlers.js

Не требуется одномоментно раскладывать весь main.js; сначала разрешён adapter layer.

24 23. Тестирование

24.1 23.1. Functional parity

- запуск/остановка всех автопилотов;
- toggle каждого чата;
- pin/remove/open;
- check now;
- settings save;
- budget rendering;
- diagnostics copy;
- logs open;
- state updates;
- ChatGPT login and navigation;
- no duplicate send.

24.2 23.2. Transition tests

- repeated click ignored;
- Esc recovers;
- transition timeout restores usable UI;
- capture failure falls back to fade;
- resize during transition;
- window minimize/restore;
- GPU process crash;
- switching chats quickly;
- ChatGPT loading state.

24.3 23.3. Visual tests

- home 16:10;
- 16:9;
- ultrawide;
- 1080p/1440p/4K;
- Windows scaling;
- reduced motion;
- high contrast;
- all chat states;
- error and budget limit.

24.4 23.4. Soak tests

- 8 hours idle;
 - 3 active chats;
 - repeated transitions;
 - memory snapshots;
 - background/foreground cycles;
 - suspend/resume.
-

25 24. Критерии готовности

25.1 Home

- выглядит как полноценная заставка без видимого dashboard;
- ровно три сферы;
- клик по каждой открывает правильный чат;
- клик по повязке открывает system hub;
- состояния чатов видны, но не создают clutter;
- idle не раздражает и не перегружает систему.

25.2 Workspaces

- настоящий ChatGPT остаётся полностью usable;
- Aegis controls доступны;
- чат нельзя перепутать;
- state обновляется без перезагрузки;
- быстрый переход не мешает работе.

25.3 Reliability

- при любой ошибке анимации остаётся рабочий интерфейс;
 - старые settings мигрируют;
 - smoke tests проходят;
 - API key не раскрывается;
 - snapshots не остаются на диске.
-

26 25. Риски и решения

Риск	Вероятность	Влияние	Ответ
WebContentsView в ПЛОХО сочетается с 3D transitions	Высокая	Высокое	отдельный capturePage POC до полной разработки

Риск	Вероятность	Влияние	Ответ
UI съедает GPU	Средняя	Высокое	профили качества, adaptive particles, pause background
Годжо выглядит дёшево	Высокая без ассетов	Высокое	layered 2.5D или pre-rendered loop, не случайная 3D-модель
Красивый UI скрывает важный статус	Средняя	Высокое	текстовые статусы, accessibility, diagnostics
Переписывание renderer ломает IPC	Средняя	Высокое	adapter layer и parity tests
Слишком долгие переходы раздражают	Высокая	Среднее	cinematic first-use + rapid subsequent mode
Generated references содержат вымышленные функции	Высокая	Среднее	использовать только композицию, привязать UI к real state
Asset licensing при публикации	Низкая сейчас	Высокое позже	private-use marker; перед релизом заменить/лицензировать

27 26. Самые смелые художественные фантазии

Ниже — moonshot backlog. Codex должен не обещать всё сразу, а классифицировать.

27.1 26.1. Полностью живой Limitless Void

- процедурный бесконечный фон;
- движение камеры зависит от курсора;
- пространственная линза вокруг Blue;

- ударные волны Red;
- область исчезновения Purple;
- система реагирует на музыку/голос опционально;
- дальние осколки формируют едва заметные символы статуса.

27.2 26.2. Six Eyes interface

- повязка раскрывается;
- глаз становится порталом;
- UI settings строится как концентрическая система;
- каждый параметр подсвечивает связанный runtime subsystem;
- ошибки показываются как дефекты в «видении» системы.

27.3 26.3. Слияние трёх чатов

Если одновременно работают три чата:

- их орбиты синхронизируются;
- Годжо меняет позу;
- между сферами появляются тонкие связи;
- общий status показывает coordinated state;
- никакого реального смешивания данных — только визуальная метафора.

27.4 26.4. Completion ceremony

Когда большая работа завершена:

- активная сфера стабилизируется;
- окружающие частицы останавливаются;
- короткий camera push;
- сфера оставляет тонкое кольцо истории;
- уведомление минимальное, без конфетти.

27.5 26.5. Error choreography

Не красный modal, а controlled anomaly:

- орбита сбивается;
- один фрагмент пространства зависает;
- текст объясняет ошибку;
- retry визуально восстанавливает геометрию;
- если recovery не удался, сцена упрощается и остаётся usable.

27.6 26.6. Cinematic history

В diagnostics можно проигрывать события как траектории трёх сфер во времени: не как игрушку, а как дополнительный обзор истории работы.

27.7 26.7. Wallpaper mode

- скрыть всю функциональность, оставить живую сцену;
 - hot corner возвращает UI;
 - auto dim;
 - понижение FPS;
 - возможно запускать как always-on-top wallpaper window, но только если это не ломает основной режим.
-

28 27. Что Codex должен ответить

Codex обязан дать ответ по следующей структуре.

28.1 A. Репозиторий

1. Какие файлы и подсистемы реально отвечают за renderer, layout, WebContentView, IPC, state и transitions?
2. Какие текущие тесты защищают поведение?
3. Какие части можно сохранить без изменений?
4. Какие parts требуют рефакторинга до визуальной работы?

28.2 B. Capability matrix

Для каждого пункта указать Да сейчас / Да после РОС / Нужен внешний ассет / Нежелательно / Нет:

- React + TypeScript renderer;
- Three.js/R3F background;
- animated Blue/Red/Purple;
- layered 2.5D Gojo;
- Live2D integration;
- full 3D Gojo;
- camera fly-throughs;
- particle systems;
- shader distortion;
- capturePage transition;
- WebContentView hide/show synchronization;
- audio system;
- contract editor;

- diagnostics timeline;
- graphics quality profiles;
- reduced motion;
- visual regression tests;
- Windows installer compatibility.

28.3 С. Честные границы Codex

Codex должен прямо написать:

- что он может создать кодом самостоятельно;
- какие изображения можно использовать как временные assets;
- какие ассеты должен подготовить художник;
- какие части требуют ручного визуального QA;
- какие эффекты слишком рискованны без прототипа;
- какие обещания нельзя дать до запуска на реальном Windows GPU.

28.4 D. Архитектурное решение

- рекомендуемый стек;
- миграционный путь;
- схема слоёв;
- state machine;
- механизм WebContentsView transitions;
- fallback path;
- security implications;
- performance strategy.

28.5 E. Первый proof-of-concept

Предложить один PR, который за 1 небольшую задачу доказывает максимум ключевых рисков.

Предпочтительная цель РОС:

Новый статичный home shell с текущим hero asset, тремя интерактивными сферами/hotspots, blindfold hotspot, route transitions и безопасным переключением между home и существующим рабочим UI без изменения runtime. Дополнительно — минимальный capturePage эксперимент за feature flag.

Codex должен перечислить:

- точные файлы;
- acceptance criteria;
- тесты;
- риски;

- что будет видно пользователю;
- что останется заглушкой.

28.6 F. План PR

Разбить работу на маленькие PR, не более одной архитектурной темы каждый.

28.7 G. Финальный вопрос

Какую максимальную версию этого визуального замысла Codex реально готов реализовать в текущем репозитории самостоятельно, сохранив стабильность Aegis, и какой первый PR он предлагает начать прямо сейчас?

29 28. Прямой prompt для Codex

Ниже можно передать Codex буквально.

Ты работаешь с репозиторием aisarus/aegis-autopilot.

Изучи весь приложенный master-спец и все PNG в assets/. Не воспринимай изображения как точные функциональные макеты: это художественные референсы. Источником истины по функциям является текущий код и его тесты.

Твоя первая задача — НЕ переписывать приложение. Проведи технический feasibility review редизайна Aegis в стиле Limitless Void с центральным Сатору Годжо, тремя интерактивными сферами Blue/Red/Purple, скрытыми рабочими экранами и киношными переходами.

Обязательно:

1. Изучи main.js, preload.js, chatgpt-preload.js, renderer/, package.json, tests/ и существующую layout/security логику.
2. Опиши точные архитектурные ограничения Electron WebContentsView.
3. Проверь идею capturePage -> snapshot animation -> hide/show WebContentsView.
4. Сопоставь реальные runtime state с визуальными состояниями.
5. Дай сарability matrix: что ты можешь реализовать кодом, что требует РОС, что требует внешнего художественного ассета.
6. Не обещай полноценного Live2D/3D-персонажа без подготовленного rig/model.
7. Сохрани все safety, budget, retry, draft protection и chat binding гарантии.
8. Предложи поэтапную миграцию renderer без большого одномоментного rewrite.
9. Предложи первый маленький PR с acceptance criteria и тестами.
10. Отдельно ответь, какую максимальную версию этого дизайна ты реально

можешь довести до работающего состояния самостоятельно в текущем репозитории.

Формат ответа:

- A. Current architecture map
- B. Hard constraints
- C. Capability matrix
- D. Recommended architecture
- E. WebContentsView transition POC
- F. Asset requirements
- G. Performance/security risks
- H. PR roadmap
- I. Proposed first PR
- J. What I can implement vs what requires external help

После feasibility review не вноси изменения, пока не сформулируешь первый безопасный PR и не докажешь, что он не ломает существующие smoke tests.

30 29. Итоговое направление

Aegis должен выглядеть не как панель, украшенная Годжо, а как **Годжо, ставший интерфейсом управления тремя чатами**.

Домашний экран — живые обои и портал. Сферы — настоящая навигация. Повязка — системный уровень. Рабочие экраны — функциональные, читаемые и основанные на реальном состоянии. Киношность строится через режиссуру камеры, физику света, частицы и умный переход между WebGL/DOM и настоящим WebContentsView, а не через бессмысленный постоянный glow.

Главный технический принцип:

Любая визуальная магия должна деградировать до полностью рабочего обычного интерфейса. Ни одна анимация не имеет права стать новой точкой отказа автопилота.